

Efficiently Combining LLMs and Humans to Grade Test Collections for Retrieval Evaluation

Candidate Number: XGJS3

Supervisor : Prof. Ingemar Cox

Department of Statistical Science
University College London

August 29, 2026

Abstract

Test collections to evaluate and benchmark retrieval systems include different queries and text passages graded by how relevant each is to the query. These relevance judgments are grades a human assessor gives to each query-passage pair. They are, however, very expensive to produce and large language models are being increasingly used to grade query-passage pairs in place of human assessors. Test collections graded by LLMs produce rankings of retrieval systems that agree closely with the ones we would obtain from a human-graded test collection. However, the agreement falls when we restrict the comparison to the best systems being evaluated. This thesis asks how to efficiently combine humans and LLMs in grading a test collection so that the obtained ranking of the best systems is close to the one a fully human graded collection would give. An LLM can report when it is uncertain about a grade, and that could be used to decide which **pairs** to send first to a trained human assessor. The best signal we built predicts the per-query agreement with human assessors at a Kendall τ of 0.34. That is not enough to decide which queries to send back to a human assessor. To find out why, we analysed what a single grade error an LLM ~~commits~~ on a passage does to the ranking of systems. The damage on the ranking can be decomposed into two parts, one measuring how wrong the grade is and one measuring how differently the systems ranked that passage. This decomposition gives a selection rule to decide which pairs should be sent first to a human assessor. **When sending only 9 to 14 percent, in four of the five years, of the pairs to a human assessor ordered by our proposed policy, the ranking of the twenty best systems agrees with the fully human ranking at a Kendall τ of 0.95.**

Contents

1	Introduction	2
1.1	Research questions and contributions	3
2	Background and related work	4
2.1	LLM judges and where they break	4
2.2	Query Performance Prediction	5
2.3	Spending a human budget	6
3	The LLM as a relevance assessor	8
3.1	Data and LLM	8
3.2	Error characteristics	9
3.3	Reliability target and evaluation measures	11
3.4	Output-based prediction	11
3.5	Query-level triage	13
4	Analysis and Correction of the Ranking of Systems	15
4.1	Decomposition of the score shift	15
4.2	The placement of errors across passages	19
4.3	Damage concentration within queries	21
4.4	Correcting the ranking at the passage level	22
5	Discussion and Conclusion	27
	References	29
A	Full Feature Inventory	31
B	Per-Year Correction Tables	32

1 Introduction

A retrieval system is a program that receives a user question, called a query, searches a collection of text passages and returns those passages as a ranked list, ordered by how relevant it thinks each one is to the query. A familiar example is a web search engine such as Google, which takes a search query and returns a ranked list of web pages. When comparing two retrieval systems, we need a set of correct answers to evaluate each system's results and determine which is better. Benchmarks used to evaluate retrieval systems, called test collections, contain many queries and, for each one, a large set of text passages with a grade from 0 to 3 saying how relevant that passage is to that query. The grades are produced by a trained human assessor who reads each query-passage pair and decides how relevant one is to the other. These grades are called relevance judgments.

When testing, every system is given the same set of queries and returns its own ranked list of passages for each one. The relevance judgments produced by trained human assessors then let us measure how well each system did. A system that places passages graded as relevant by the assessor near the top of its list scores better than one that places text passages graded 0 there. Each system therefore receives a single number summarising how good its results were across all the queries and ordering the systems by that number gives what we call a ranking of retrieval systems.

An example of one query and two passages is given in Table 1. Each combination of the query with one passage is a query-passage pair and a trained human assessor graded each of the two.

Query	do goldfish grow
Grade 3	The conditions goldfish are kept in plus their diet determine how large they will grow. I have seen goldfish grow ridiculously large in very small containers when their water was changed frequently. Goldfish will not grow if water conditions are poor. Fancy goldfish don't grow as large as Common goldfish. A good size would be around 5 inches body length for most fancy varieties, 8 inches for Comets and 12 inches for Common Goldfish. These sizes are usually only attained by pond grown fish.
Grade 1	All Koi have the same body type, but very greatly in color. They can also grow up to two to three feet in length. Most goldfish are orange in color, with a few types that also have black, white and calico colors. They come in a wide variety of body shapes, but stay relatively small in size.

Table 1: One query and two passages from TREC Deep Learning 2019, with the grades assigned by a trained human assessor from NIST (National Institute of Standards and Technology). The second passage mentions goldfish and their size, but does not answer whether they grow.

However, relevance judgments are expensive to obtain. For each pair query-passage, a trained assessor has to read the query, the passage and carefully decide how relevant the passage is to the query. When multiplied for thousands of pairs the costs increase ~~considerably quick~~. For this reason, researchers are trying to automate relevance scoring using large language models (LLMs). If an LLM is able to produce relevance judgments with the same quality as a human assessor, the process would become cheaper and faster, benefiting the whole field.

Surprisingly, it already works reasonably well. If you build two rankings of retrieval systems,

one using a test collection graded by LLMs and another using the same test collection graded by human assessors, the two rankings align with a rank correlation close to 0.89 (Upadhyay et al., 2024b).

Using LLMs instead of humans for evaluation raises three concerns. First, Clarke and Dietz (2025) studied what happens when we restrict the comparison to the systems ranked highest by human assessors and found that the ranking agreement drops to a correlation of 0.51, raising serious doubts about the reliability of using LLMs. Second, LLMs show a systematic bias towards giving higher grades than human assessors. Lastly, having a machine grade a test set creates a circularity problem. A system could now optimize for whatever the LLM happens to reward rather than exactly what a human would value.

The field has been searching for hybrid methods, where an LLM is used to reduce the human effort. One idea is to keep the LLM grades on a portion of the query-passage pairs of the test collection and send difficult or more important pairs for a human to grade. ~~One idea is to keep the LLM grades on a portion of the query-passage pairs and send the difficult or more important ones for a human assessor to grade.~~ LARA, for example, spends a human budget on judgments the LLM claims to be uncertain about (Takehi et al., 2025).

The ultimate goal is that the ranking of systems built from the final produced grades matches the ranking that fully human grades would give. The aim is to reproduce that ranking while sending as few pairs as possible to a human assessor.

This thesis asks one question: when grading a test collection, what information should be used to decide where the human effort goes?

The first idea is that the LLM itself can provide this information. ~~Just~~ like Takehi et al. (2025) ~~did~~, one can look at the grades the LLM produced and at how certain it was about each one, and ask whether these are enough to tell us which grades can be trusted.

Another idea is that what matters is not only how wrong the LLM is, but how relevant that passage is to distinguish between retrieval systems. ~~The intuition behind is that~~ even if an LLM fails in one passage, it only damages the ranking of systems if two systems disagree on how to rank this passage.

1.1 Research questions and contributions

The three proposed research questions follow from the above stated ideas.

- RQ1: Can the LLM report its own uncertainty when grading one query-passage pair?
- RQ2: If we know how well the LLM agrees with the human assessors on a query, is that enough to choose which queries to send to a human assessor?
- RQ3: **What does protecting the ranking of systems** actually require and how much human budget is it needed to correct the ranking?

On the first research question, we analyse the LLM’s grades and its reported confidence, and what these tell us about how well it agrees with the human assessors. The best signal we construct from them predicts that agreement at a Kendall τ of 0.34, a metric of how well two rankings agree.

Our second contribution explains why sending whole queries to a human assessor, chosen by how well the LLM agreed with the human assessors on them, does not keep the ranking of systems correct. The reason is that a small number of passages in every query are responsible for the **damage** created in the ranking of systems. We also show that an LLM grading error only moves the ranking if the systems being compared rank that passage differently.

The last contribution compares different methods to decide which pairs to send to a human assessor in order to **preserve the ranking of systems**. After analysing what damages the ranking of systems, we propose a policy to choose which pairs to send to the human first. Sending only 9 to 14 percent of the pairs to a human assessor in four of the five collections, and 28 percent in the fifth, produced a ranking of the twenty best systems that agrees with the fully human ranking at a Kendall τ of 0.95.

Section 2 covers the existing literature and related work, while section 3 describes the data, the LLM and the patterns of its errors. Then tests whether the LLM can report when it is uncertain about a specific grade. Section 4 analyses how one single error affects and damages the ranking of systems, and proposes a method to preserve the top of the ranking with a low human effort. Section 5 discusses what the thesis explains and proposes, and what cannot be claimed.

All code used in the thesis can be found in the following GitHub repository: <https://github.com/jbarao04/when-can-an-llm-judge-be-trusted>.

2 Background and related work

2.1 LLM judges and where they break

Two rankings of retrieval systems can be built, one scored with human grades and one with LLM grades. When considering all systems submitted in a year, the two rankings are close. **This is less surprising than it looks. The score of a system is an average over all the queries of that year, so errors on single passages tend to cancel out.** Upadhyay et al. (2024a) compare the two rankings on the five TREC Deep Learning collections used here and report Kendall τ between 0.87 and 0.94. Faggioli et al. (2023) report 0.86 on TREC DL 2021 with GPT-3.5.

The agreement between the rankings falls when the comparison is limited to the strongest retrieval systems. These are also the ones that matter the most to distinguish, since test collections are used by researchers and by industry to find out which systems work best, and that decision is made at the top of the ranking. Clarke and Dietz (2025) tested this restricted to the twenty best systems. The twenty are chosen from the human ranking, since those are the systems that would actually be compared. We then take those same twenty systems, order them by the scores they receive under the LLM grades, and compare these two rankings of twenty systems. Systems outside the top twenty are ignored entirely. The agreement falls from 0.89 over all systems to 0.51 over the top twenty. **To give some intuition, if two of those twenty systems are picked at random, the ranking built from LLM grades places the worse one above the better one almost a quarter of the time.**

Clarke and Dietz (2025) raise a second concern, about what happens once such grades are in use. If a machine produces the grades, and everyone knows which machine and which prompt were used, then a retrieval system can be built to score well against that machine. To test this, they built a system in the simplest possible way. Instead of using the LLM as a judge or human assessor, they used an LLM as the retrieval system itself. The point is that the evaluation also

grades passages using that exact LLM model. As it would be expected, the system performed extremely well and scored fifth out of seventy-five. Using human assessors grades the system was ranked twenty-eighth. It did not reach the first place, because even though the same model was used, the prompt used was different.

The authors then ask what would happen if all participating teams adapted and submitted systems like this. They take each submitted system and re-order its passages by the LLM grades, so that every system now ends with the LLM as its last step. These new systems are scored twice, once with human grades and once with LLM grades, and the two rankings are compared. Over the top twenty systems the agreement falls to 0.44, while over the top five it is -0.40 , so the two rankings are close to reversed. The LLM grades can no longer evaluate and distinguish between the systems, because every system now agrees, by construction, with the LLM. Balog et al. (2025) report a related bias in favour of systems that use an LLM.

Even before ~~the generative AI revolution that~~ brought LLMs, less accurate grades were known to affect the ranking of systems. Soboroff et al. (2001) tried to leave human grades aside and graded random pairs query-passage as relevant. Under those new grades the best systems scored like the worst ones. The reason is that a passage drawn at random is likely to be one that many retrieval systems retrieved, so the procedure rewards a system for returning what others also return (Aslam and Savell, 2003). The systems that lose are the ones that find unique passages the others missed.

In 2008, Bailey et al. (2008) tried to change the assessors instead of the grades. In this work, they compared three groups: the first group wrote the queries themselves and worked in the field the queries came from; the second group worked in the same field but did not write the queries; while the third one did neither. The three groups then graded all pairs. Of the pairs query-passage the third group called highly relevant, the first group agreed on 58% and called 24% not relevant at all. The ranking of systems built from the third group’s grades reaches Kendall τ of 0.66 against the ranking from the first. In the same comparison, the second group reaches tau of 0.94. The second, just like the third, did not write the queries, so if the drop came from simply asking someone that had not written the queries, both would show it. Their conclusion is that what the human assessor knows about the query topic is what actually moves the ranking.

In a position paper, Soboroff (2025) lays out a strong case against using LLMs to grade test collections. The argument says that whatever produces the correct grades sets a ceiling on what the evaluation can measure. This happens because no system can score better than what made the grades. Consider a state-of-the-art LLM used to produce the grades. The next version of the same LLM, submitted as a retrieval system, would be measured as equal or worse than its predecessor and could never beat it. One exception is stated: an LLM can produce the grades if it holds relevance information the systems being measured do not have. Our proposed method studied in Section 4.4 fits partly inside this exception and Section 5 returns to this later.

The difficulty of distinguishing the systems at the top of the ranking, where they are closest to each other, has been documented for more than twenty years.

2.2 Query Performance Prediction

An important idea introduced in Section 1 is that an LLM can tell us when it is more uncertain about a certain pair. One can expect the LLM to be less reliable and more likely to produce a wrong grade, when it is uncertain.

Query performance prediction (QPP) is an older topic, that asks the same question about a different object. Given only the query, can we predict how well a retrieval system will perform on it? A retrieval system is considered to perform well if it is able to retrieve the most relevant passages as its top results. QPP looks for signals that indicate whether the system has retrieved relevant passages for that query, using only the query and the results the system returned. As an example, if the top results are all similar to each other and different from the rest of the collection, the system has probably found something specific. If they look like any other passages in the collection, it probably has not.

In Cronen-Townsend et al. (2002), they compare the words used in the top retrieved passages against the words in the whole collection of text passages. If the top retrieved passages by the system use the same language as the collection as a whole, the query has probably not found anything relevant.

First Rank Change (Zhou and Croft, 2007) adds noise to the passages and then checks whether the system still returns the same first. If the system is consistent, one can expect it to still retrieve the same passages, even when some noise is added.

Vinay et al. (2006) built four signals, from the set of retrieved passages, that predict the performance of the system. Each one is meant to show whether the retrieval worked. The one that obtained the best results computed how stable the ranking is when a text passage is perturbed. It reaches Kendall $\tau = 0.521$ against average precision, followed by clustering tendency at 0.441, local intrinsic dimensionality at 0.268 and the stability under query perturbation at 0.174. Three of the four combined reach 0.562. The perturbation features from Section 3.4 are built on the same idea, which is to create a perturbation and see whether the output holds.

2.3 Spending a human budget

There are different methods that can be used to reduce human effort in grading test collections using LLMs. These differ in how they divide the work with the human. Four lines of work are presented, but only one of them produces a test collection that a new system built later could be scored against.

In the first, the human assessor grades a sample of pairs and the same sample is used to measure the LLM. Merlo et al. (2025) do this by picking a pair at random, sending it to a human assessor and using the grade it gave to update an estimate of the agreement between the LLM and the human. They repeat this process until the estimate is accurate enough. Unell et al. (2026) instead chooses a sample whose agreement matches that of the full set of pairs, and accepts a small bias in exchange for a smaller error at low budgets. These works do not change grades in the collection, but they improve the estimate of how good the LLM is, which is not the problem this thesis works on.

In the second line of work, a machine grades the pairs the human never graded. It is important to remember that a human assessor only grades the pairs that were ranked in the top-10 of a submitted system or considered highly relevant. This means that all other passages left are considered not relevant. In this context, a machine can be an LLM, but also a different model trained for the task. MacAvaney and Soldaini (2023) simulate a test collection where almost no pairs have been graded. For each query they keep only one passage the human assessors graded as relevant and treat every other passage as ungraded. A machine is used to predict the grades of those passages. The machine grades are poor and the authors state that they are not good enough to be used as automatic grades. However, the ranking of the submitted systems still holds at Kendall τ between 0.87 and 0.92 against the ranking the full human grades give.

In the third, the LLM helps the human instead of replacing them. This line is underexplored and the two papers below only propose ways of doing it, but neither tests them. Faggioli et al. (2023) refers to this line of work as the most credible path and suggests that an LLM could be used summarise a passage before a human reads it. Another idea is to use an LLM to produce a first grade with an explanation attached that the human assessor then uses to help grading. Soboroff (2025) suggests a similar idea. In his proposal, the model watches the human assessor and flags possible mistakes. One warning is raised: humans are known to trust a suggestion from a machine more than they should, which could introduce a bias in the human grades that would not exist without the machine.

In the fourth line, the LLM grades every pair and the human assessor corrects a chosen subset of pairs. What comes out in the end is a mix of human and LLM grades, that can be used to score and rank new retrieval systems. This is the line pursued by this thesis and it is where the closest published work exists.

LARA, or LLM-Assisted Relevance Assessments (Takehi et al., 2025), takes the probabilities outputted by LLM when choosing a grade. These probabilities are not calibrated, so LARA fits a small model that maps the reported probability to how often that grade is actually correct. After this, LARA chooses and sends pairs to the human assessor **by the gap between the two most likely grades**. The idea is that a small gap, for example the LLM giving 0.45 to grade 1 and 0.40 to grade 2, means the LLM could not separate the two grades, so a human is most useful there. If instead the LLM gives 0.95 to grade 3, it is unlikely to be wrong and a human is not needed. Other work also reads what the LLM produced, rather than only the grade it settled on. Wang et al. (2025) compares the grade the model settles on against the average grade **under its own probabilities, and the average agrees better with human preference in 42 of 48 settings**.

Other works ignore the LLM and read the runs of the submitted systems instead. Some passages can be more important than others to distinguish between systems, so it can be worth sending these to human assessors. Move-to-front (MTF) pooling gives every submitted system a priority and takes the next passage to send human assessors from the system at the top (Cormack et al., 1998). If that passage is graded as relevant, the same system chooses the next one. If it is not, the system drops down the priority list. Minimal test collections read the gap between two systems directly (Carterette et al., 2006). That gap is a weighted sum over the passages, where the weight depends on where each system placed the passage. A passage both systems placed at the same rank has no weight, so its grade cannot change which system wins. The method judges the passages with the largest weights first.

Two signals are therefore available for deciding which pairs a human should correct. The LLM says where its own grade may be wrong and the submitted runs can say which passages are more important to distinguish between systems. If we only focus on correcting the grades the LLM is least sure about, we are treating every error as equally important and this is not correct.

From this literature, this thesis studies how to efficiently combine LLMs and humans when grading test collections to evaluate retrieval systems. Following the methods introduced by Takehi et al. (2025) and Wang et al. (2025), and taking inspiration from query performance prediction, we try to understand when the LLM is unsure about its grade, using the probabilities it reports.

We then analyse how a single grade error can damage the ranking of systems compared to the ranking the human grades give. From this we obtain some intuition about which passages affect the ranking most and, finally, design a method to send a selected number of pairs to a human assessor.

3 The LLM as a relevance assessor

3.1 Data and LLM

The data is from the TREC Deep Learning passage-ranking task, which ran for five years, from 2019 to 2023 (Craswell et al., 2020, 2021, 2022, 2023, 2025). Each year provides a set of queries and, for each query, a pool of text passages graded by NIST assessors on a 0 to 3 relevance scale. The task for retrieval systems is to rank passages by relevance to the query. TREC also makes available, for each query, the ranking of passages that each retrieval system submitted. These rankings are called runs, so a run is one system’s ranked list of passages for every query in a given year. An important note is that in 2023 the queries were not all written by humans. Alongside 51 human queries, the organisers generated 31 synthetic ones with a fine-tuned T5 model and a GPT-4 prompt.

A passage corpora is the collection of text passages a retrieval system searches when it receives a query. Over the five years, two different passage corpora were used. TREC DL years 2019 and 2020 use MS MARCO v1, that contains 8.8 million passages, and years 2021 through 2023 use MS MARCO v2, that contains 138 million passages. Assessors cannot grade all the passages from the corpus. For each query, NIST collects the top ten passages from each run and finds additional relevant passages that no system ranked in its top-ten, using a model trained on the grades already produced that ranks the ungraded passages by how likely they are to be relevant. Human assessors grade this pool of passages.

The five yearly collections, from 2019 to 2023, contain in total 308 queries, 71,956 graded query-passage pairs and 294 submitted runs. The number of systems submitted is not the same every year, and one team of researchers can submit close variants of the same system. The yearly totals range from 35 in 2023 to 100 in 2022.

Years 2021 to 2023, which use MS MARCO v2, require a pre-processing step. MS MARCO v2 contains large numbers of passages that are almost identical to each other. TREC groups them and gives every passage in a group the same grade, so that two passages with the same text never receive different grades. In 2022 this inflated the number of graded pairs to 386,416 for 76 queries, an average of 5,084 per query. NIST makes available a file that defines the groups and names a representative for each. We keep only the representative passage of each group and drop the rest, which reduces the number of graded pairs in 2022 by 93.9%. The same procedure was applied to 2021 and 2023.

Year	Corpus	Queries	Graded pairs	Submitted runs
2019	MS MARCO v1	43	9,260	37
2020	MS MARCO v1	54	11,386	59
2021	MS MARCO v2	53	8,910	63
2022	MS MARCO v2	76	23,522	100
2023	MS MARCO v2	82	18,878	35
Total		308	71,956	294

Table 2: Dataset composition by year. Graded pairs are counted after removing duplicated in v2.

The LLM model used is Llama-3.1-8B-Instruct. Each query-passage pair is graded with the UMBRELA (UMbrela is the Bing RELevance Assessor) prompt (Figure 1), an open-source implementation of the grading procedure Bing uses in production (Upadhyay et al., 2024a; Thomas et al., 2024). The prompt asks the LLM for a grade on the same 0 to 3 scale that

Given a query and a passage, you must provide a score on an integer scale of 0 to 3 with the following meanings:

- 0 = represent that the passage has nothing to do with the query,
- 1 = represents that the passage seems related to the query but does not answer it,
- 2 = represents that the passage has some answer for the query, but the answer may be a bit unclear, or hidden amongst extraneous information and
- 3 = represents that the passage is dedicated to the query and contains the exact answer.

Important Instruction: Assign category 1 if the passage is somewhat related to the topic but not completely, category 2 if passage presents something very important related to the entire topic but also has some extra information and category 3 if the passage only and entirely refers to the topic. If none of the above satisfies give it category 0.

Query: {query}
Passage: {passage}

Split this problem into steps:
Consider the underlying intent of the search.
Measure how well the content matches a likely intent of the query (M).
Measure how trustworthy the passage is (T).
Consider the aspects above and the relative importance of each, and decide on a final score (O). Final score must be an integer value only.
Do not provide any code in result. Provide each score in the format of: ##final score: score without providing any reasoning.

Figure 1: The UMBRELA prompt (Upadhyay et al., 2024a). Each query-passage pair is graded with this prompt, with the output constrained to a single token from $\{0, 1, 2, 3\}$.

human assessors use.

LLMs have one important parameter that should be mentioned. Temperature controls how the model chooses its output. A lower temperature results in a more predictable output, while a higher temperature obtains outputs less expected and more unpredictable. We choose temperature 0, so the LLM emits the grade with the highest probability rather than sampling one from the four.

Additionally, when an LLM answers a prompt, it does not just pick its output directly. It first computes a probability for every output it could give and then returns the most likely one. In our case, the answer is a single grade, so the LLM produces one probability for each of the four grades. As we are working with an open-source model, we have access to them and keep the four probabilities for each graded pair.

3.2 Error characteristics

The LLM scored all 71,956 pairs across the five years. We start by comparing the LLM grades with the human assessor ones for each query-passage pair.

Spearman correlation between the two sets of grades is 0.444. It is the Pearson correlation of Equation 3.2 applied to the ranks of the grades rather than the grades themselves, so it measures whether the two sets order the passages the same way and ignores the size of each difference.

Quadratic-weighted kappa is 0.347. Let O_{ij} be the number of pairs the LLM graded i and the human graded j , and E_{ij} the number expected if the two graded independently, obtained from how often each of the two uses each grade. Weighting each cell by $w_{ij} = (i - j)^2/9$, which is

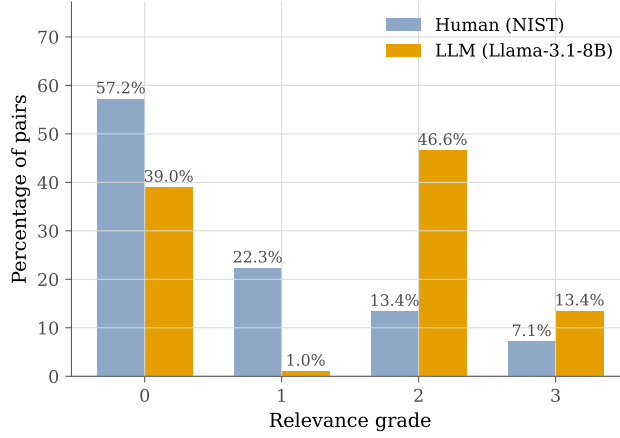


Figure 2: Grade distribution across all 71,956 judged pairs. The LLM barely uses grade 1, compressing the effective scale from four points to three.

zero when the two agree and one when they are three grades apart, so that larger disagreements count more,

$$\kappa_w = 1 - \frac{\sum_{ij} w_{ij} O_{ij}}{\sum_{ij} w_{ij} E_{ij}} \quad (3.1)$$

A value of 1 means the two always agree and 0 means they agree no more than by chance.

The LLM emits the same grade as the human on 43.1% of pairs and a grade within one point of the human on 69.8%. These numbers describe average pair accuracy, but do not show whether the errors the LLM commits change how retrieval systems are ordered relative to one another once the LLM grades are treated as correct.

LLM judges are often reported to assign higher grades than human assessors (Alaofi et al., 2024), although the direction can vary with the model (Section 2.1). This LLM over-scores 46.3% of the 71,956 pairs, matches the assessor on 43.1% and under-scores 10.6%. Across all pairs, the LLM grades are on average 0.64 points higher than the human grades. Another relevant pattern is that humans assign grade 1, “on topic but does not answer”, to 22.3% of passages, while the LLM assigns it to only 1.0%. In contrast, LLMs assign grade 2 to 46.6% of the passages, while humans use it on 13.4%.

The agreement between human assessors and LLM also varies across queries. For each query q with n_q graded query-passage pairs, we compute the Pearson correlation between the LLM grades and the human grades:

$$r(q) = \frac{\sum_{i=1}^{n_q} (g_i^{\text{LLM}} - \bar{g}^{\text{LLM}})(g_i^{\text{human}} - \bar{g}^{\text{human}})}{\sqrt{\sum_{i=1}^{n_q} (g_i^{\text{LLM}} - \bar{g}^{\text{LLM}})^2 \sum_{i=1}^{n_q} (g_i^{\text{human}} - \bar{g}^{\text{human}})^2}} \quad (3.2)$$

where g_i^{LLM} and g_i^{human} are the LLM and human grades for the i -th passage of query q . This correlation ranges from -0.13 to 0.81 across queries, with mean 0.40 and standard deviation 0.17 .

Queries with low agreement, where the human grades and the LLM grades differ most, look like the obvious place to send to a human assessor. If the human assessor received the queries with the lowest agreement and graded every pair in them, the test collection would have human grades precisely where the LLM did worst. Section 3.5 tests this idea and shows why it fails.

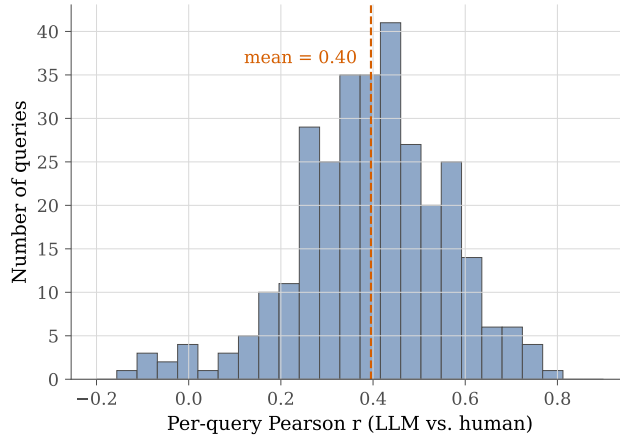


Figure 3: Per-query Pearson correlation between LLM and human grades.

3.3 Reliability target and evaluation measures

Before we can ask what makes the LLM grading reliable and good on a query, we need one number that says how close its grades came to the human grades on that query.

The chosen primary target is the per-query Pearson correlation of Equation 3.2. It measures whether the LLM separates relevant from non-relevant passages in the same way the human assessors do, and it uses the size of each grade difference, not only the order.

Another candidate target was per-query nDCG@10. It would rank each query’s passages by the LLM grade, then score that ranking using the human grades as relevance:

$$\text{nDCG@10}(q) = \frac{\sum_{i=1}^{10} \frac{g_{\pi(i)}^{\text{human}}}{\log_2(i+1)}}{\sum_{i=1}^{10} \frac{g_{(i)}^{\text{human}}}{\log_2(i+1)}} \quad (3.3)$$

where $\pi(i)$ is the passage the LLM places at rank i , and $g_{(i)}^{\text{human}}$ is the i -th largest human grade for that query, which gives the best ranking possible.

This target was rejected, because the LLM assigns only four distinct grades to the passages. Many passages therefore share the same grade and so which of them reach the top ten is decided by the tie-breaking rule, and not by the LLM.

3.4 Output-based prediction

When deciding which queries to send ~~first~~ to a human assessor, we do not have the human grades, so we cannot sort the queries by the per-query Pearson correlation. A goal of this thesis is therefore to find signals in the LLM’s own outputs that track that correlation. These signals can be extracted from the grades the LLM gave or from the probabilities it reported for each grade. A simple example is the probability the LLM puts on the grades it chooses for the pairs of one query. If it consistently puts 0.90 of the probability on the chosen grade and spreads the remaining 0.10 across the other three, it looks more confident than if it spreads the probability equally across all four grades. We call these signals ~~red~~ features and each one produces a single

value per query. Recall that the purpose of a feature is to ~~say~~ how reliable and confident the LLM was when grading the pairs of that query. A good feature is one that is highly correlated with the per-query Pearson correlation, so that the queries it scores lowest really are the ones where the LLM agreed least with the human assessors.

To find the best features, we rank all queries by the feature and by the per-query Pearson correlation, then measure how well the two rankings agree ~~with~~ Kendall τ . Kendall τ is computed by taking every pair of queries. A pair is concordant if both rankings put the two queries in the same order and discordant if they disagree. With n queries and $\binom{n}{2}$ pairs in total:

$$\tau = \frac{C - D}{\binom{n}{2}} \quad (3.4)$$

where C is the number of concordant pairs and D the number of discordant pairs.

We developed and tested two types of features. *Static* features read the grades and probabilities the LLM already produced and look for a pattern in them that signals reliability. *Perturbation* features look at how the LLM’s grade changes when we alter the query, the passage or the prompt. One could expect a reliable LLM to hold its grade steady under a small rewording and ~~an unreliable one to move~~.

We tested more than fifty static features ~~ad~~ the two strongest use the expected grade of a passage, the average grade under the LLM’s own probabilities.

$$s_p = \sum_{g=0}^3 g \cdot P(g \mid p)$$

The best feature is the **Fisher discriminant ratio** at $\tau = 0.297$. This feature splits a query’s passages into those the LLM graded low ($Grade = \{0, 1\}$) and those it graded high ($Grade = \{2, 3\}$) and then asks how far apart the two groups sit relative to how spread out each one is

$$F_q = \frac{(\mu_H - \mu_L)^2}{\sigma_H^2 + \sigma_L^2}$$

with μ and σ^2 the mean and variance of s_p inside each group. **The idea is that an LLM that has cleanly separated relevant from irrelevant should agree with humans more often.**

The second best static feature, at $\tau = 0.281$, is the standard deviation of the expected scores across the n_q passages of query q :

$$\sigma_q = \sqrt{\frac{1}{n_q} \sum_{p=1}^{n_q} (s_p - \bar{s})^2} \quad (3.5)$$

A high σ_q means the LLM gives the **passages of that query clearly different expected scores.** A low σ_q means the expected scores are close together and the LLM separates the passages weakly.

We also tried three perturbation operators: swapping words for synonyms, shuffling the sentences of the passage and rephrasing the prompt instruction. Swapping words for synonyms and rephrasing the instruction reached, respectively, $\tau = 0.169$ and $\tau = 0.259$, both below the best static feature. We therefore focus only on the third, and report the other two in Appendix A.

Sentence shuffling reorders the sentences of each passage and scores the shuffled version, three times with a different random seed each time. Passages with fewer than two sentences cannot be shuffled and are left unchanged - 28.3% of passages fall in this category. Each time we rank the query’s passages by expected score and compare that ranking with the unshuffled one using Kendall τ_b . The feature is the mean of the three τ_b values and it reaches $\tau = 0.338$, higher than Fisher score. This operator needs, however, three extra LLM scores per passage, because it requires the LLM to grade three reshuffled versions of the passage.

A normal objection is that a bigger LLM model would be better at reporting its own confidence on the scored passage. To check, we repeated the scoring with the an LLM model Qwen2.5 at four parameter sizes, from 0.5 Billion Parameters up to 7 Billions, and recomputed the features each time. The best feature stays near $\tau = 0.37$ at every model size, growing the model by a factor of fourteen shows no systematic change. At the same time, this is only evidence within one family of open-source models and says nothing about what a commercial LLM such as Anthropic’s Opus-5 would do.

3.5 Query-level triage

The following experiment uses the features and tests if these can be used to decide which queries should be send to a human assessor, and which we can trust an LLM. The goal is to to build a test collection, with some queries graded by the LLM and other by human assessors.

Every retrieval system submitted a ranked list of passages for each query. To score a retrieval system we use nDCG@10, the same metric used by TREC, which reads only the ten passages the system placed at the top. For one query q , let $\pi(i)$ be the passage the system placed at rank i , and let g_p be the grade of passage p :

$$\text{nDCG@10}(s, q) = \frac{\sum_{i=1}^{10} \frac{g_{\pi(i)}}{\log_2(i+1)}}{\sum_{i=1}^{10} \frac{g(i)}{\log_2(i+1)}} \quad (3.6)$$

where $g_{(i)}$ is the i -th largest grade available for that query, so the denominator is the best score any ranking could reach. The score of a system is its average over the Q queries of that year:

$$\text{nDCG@10}(s) = \frac{1}{Q} \sum_{q=1}^Q \text{nDCG@10}(s, q) \quad (3.7)$$

Sorting the systems by this score gives the ranking of retrieval systems. The ranking produced from the fully human graded test collection is what we want to preserve when we use the same collection graded by a combination of humans and LLMs.

It is important to understand that when the same retrieval systems are evaluated under the same test collection with different grades for the pairs, the systems may end up ordered differently. o understand if the test collection graded by LLMs and humans produces a reliable ranking, we measure the similarity between the ranking of systems produced by the fully human graded collection and the ranking produced by the mixed collection.

The human budget is the fraction of queries we send to human assessors. For each of these

queries, human assessors grade every query-passage pair associated with that query. At a budget of 0%, every grade in Equation 3.6 comes from the LLM. At a budget of 100%, every grade comes from a human, which gives us the original ranking we want to preserve. Between 0% and 100%, the grades are mixed: some queries use human grades, while the rest use LLM grades. For example, a 30% budget means that the 30% of queries chosen by the feature are graded by a human, and the remaining 70% keep their LLM grades.

The feature sets this order by sorting queries from least to most reliable. It corrects the least trusted queries first, leaving the most trusted for last. We then increase the budget from 0% to 100%, rebuilding the ranking of systems at each step and comparing it to the original ranking using Kendall τ (Equation 3.4).

We report Kendall τ to compare both rankings, but also a different metric called Kendall $\tau@20$, following Clarke and Dietz (2025), who show that LLM grades order the top systems much worse than they order the full ranking. This metric uses only the twenty systems that the reference ranking places highest. ~~The difference is that~~ at every budget, we order these same twenty systems using the **mixed test collection** and compute Kendall τ between that order and the order they have in the reference ranking.

We compare the two features against a random selection baseline that simply picks queries at random and sends them to the human assessor. **At each budget, we draw 1000 random subsets of queries of that size, use human grades for those queries, and build the ranking of systems for each draw.** We then compute the mean Kendall τ **and a 95**

In Table 3, we present Kendall τ and $\tau@20$ at budget zero, when every grade comes from the LLM and no query has been corrected. It reports the Kendall τ between the ranking of systems built from the LLM grades and the reference ranking built from the human grades.

	2019	2020	2021	2022	2023
τ_{all}	0.931	0.895	0.860	0.883	0.852
$\tau@20$	0.874	0.653	0.684	0.653	0.726

Table 3: System-ranking agreement with the human ranking when the LLM grades every pair.

Neither the Fisher discriminant ratio nor the sentence-reshuffling stability features perform better than the random selection baseline. Table 4 shows, for each year, the area between each feature’s curve and the random curve on $\tau@20$. For both features the values are near zero and even negative in some years, indicating that the ordering of queries they produce is not better than random.

We also tested an oracle, ~~which we call the reliability oracle~~, that sorts queries by their per-query Pearson correlation with the human grades. **It clearly could not be used as a policy to choose which queries to send first to a human assessor, since it uses the human grades themselves, which are not known until the queries have already been sent.**

The oracle’s area above the random mean is between +0.008 and +0.043. The value is positive in all five collections, so the ordering is slightly better than random. **A small positive area could come by luck from the particular queries in that collection. It could also come from the particular ordering, since some orderings do better than others by chance alone.**

To see how much the **result** depends on the particular queries, we use a **percentile bootstrap** over queries. We draw Q queries **with replacement** from the Q queries of the collection. We re-run the full procedure on that sample: we sort its queries by their Pearson correlation with the human grades, raise the human budget from 0% to 100%, rebuild the ranking of systems at each budget level, compare it with the reference ranking of the same sample, and record the area

against the random curve. Repeating this 1000 times gives 1000 area values and the interval between their 2.5 and 97.5 percentiles is the bootstrap confidence interval for the area. This interval contains zero in all five collections, on τ_{all} and on $\tau@20$. The measured improvement is therefore consistent with the ordering giving no improvement at all on a different set of queries.

The second test keeps the queries fixed and generates 1000 random orderings, that create 1000 curves. At each budget level these curves cover a range of τ values, and we take the 2.5 and 97.5 percentiles of that range. The oracle’s curve stays inside this range at every budget level, in every collection, on both τ_{all} and $\tau@20$. Ordering the queries by the per-query Pearson correlation therefore produces a curve that a random ordering of queries chosen at random could also produce.

Area improvement over random, $\tau@20$	2019	2020	2021	2022	2023
Sentence-reordering stability	+0.003	−0.013	+0.026	−0.011	−0.003
Fisher discriminant ratio	+0.012	+0.013	+0.044	−0.008	−0.030
Reliability oracle	+0.043	+0.025	+0.008	+0.021	+0.010

Table 4: Area between each query-ordering curve and the random curve, on $\tau@20$, per collection. Intervals come from resampling queries and all of them include zero.

We tested also an even stronger oracle, which we call the damage oracle, that measures how much each query damages the ranking of systems on its own. For one query, we switch that query alone from human to LLM grades, keep every other query on human grades, recompute each system’s mean nDCG@10 and re-rank the systems. The damage of that query is $1 - \tau$ between this ranking and the reference ranking, so a larger value means the query moved the systems further out of order.

We then order the queries from least to most damaging, so the queries the LLM handles most safely stay with the LLM. This ordering does better than the first oracle in four of the five collections, which is expected, because it is sorted by the outcome it is being scored on. It is still small, since the area above the random mean is between +0.001 and +0.076 and every bootstrapped confidence interval covers zero.

Figure 4 shows the curves against the random baseline. The grey band covers the 2.5 to 97.5 percentile of the 1000 random orderings, so a curve that stays inside it is not distinguishable from picking queries at random.

This is a null result. We could not find a way of selecting which queries to send to a human assessor that works better than sending them at random. In Section 4, we analyse why this fails and propose a different approach of efficiently grading a test collection using LLMs and human that produces a ranking of systems very close to the original ranking, while having humans grade only a small portion of the pairs.

4 Analysis and Correction of the Ranking of Systems

4.1 Decomposition of the score shift

In this section we look at what happens when the grades in the test collection change from human grades to LLM grades, and how that affects each system’s score and the ranking of systems. Let us first define the score that ranks retrieval systems in the TREC setting. For system s and query q , let $p_{s,i}$ be the passage ranked at position i and $g(p)$ the grade treated

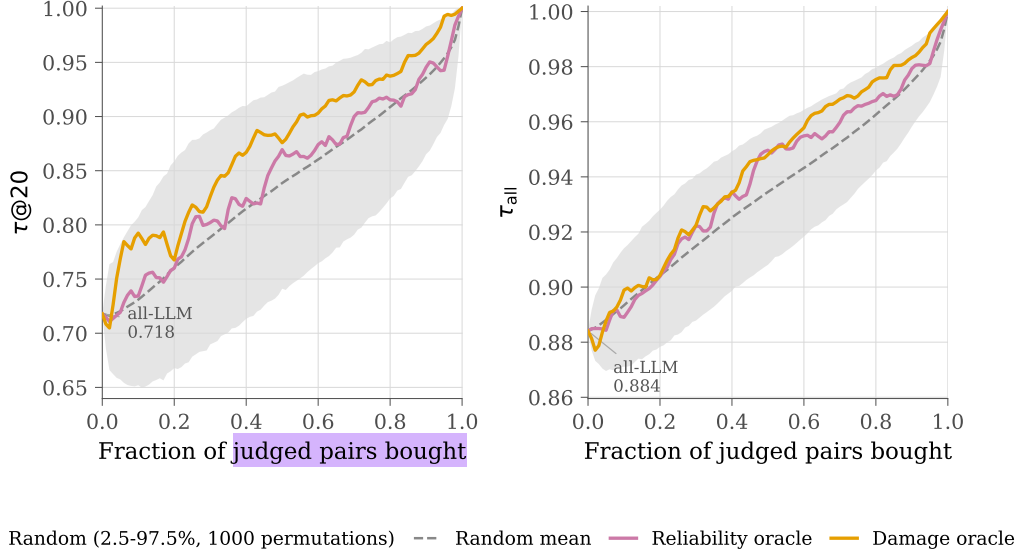


Figure 4: Query-level selection against random query orderings, averaged across the five collections. Left: agreement over the twenty systems the human grades rank highest. Right: agreement over all systems. The two curves order the queries by their Pearson correlation with the human grades and by the damage each query causes on its own. The band is the 2.5 to 97.5 percentile of 1000 random orderings of the same queries. Mean area above the random mean on $\tau@20$ is +0.021 for the Pearson ordering and +0.039 for the damage ordering. All intervals come from resampling queries and include zero.

as correct for that passage. As we have seen before, the score of a system is the average of its scores across the queries of that year. The score for one query alone is defined using a known metric, $nDCG@10$. It is the ratio between the score of the ranking the system produced and the score of the best ranking possible for that query.

$$DCG_s(q) = \sum_{i=1}^{10} \frac{g(p_{s,i})}{\log_2(i+1)}, \quad Z(q) = \sum_{i=1}^{10} \frac{g(i)}{\log_2(i+1)} \quad (4.1)$$

The first sum reads the ten passages system s placed at the top. Each passage contributes its grade, discounted by how far down the list it sits, so a relevant passage at rank one counts for more than the same passage at rank ten. The second sum is the largest value the first could take on that query, obtained by placing the highest graded passages first, where $g_{(1)} \geq g_{(2)} \geq \dots$ are the grades of that query sorted in descending order.

Dividing one by the other puts every query on the same scale. Without it, a query with many relevant passages would allow a higher score than a query with few, whatever the system does. The score of system s on query q is then

$$nDCG_s(q) = \frac{DCG_s(q)}{Z(q)} \quad (4.2)$$

The numerator of each term in the DCG sum is the gain of the passage. Written as above it is the grade itself, called linear gain. TREC instead uses exponential gain, replacing g with $2^g - 1$. For the rest of this thesis, we will use linear gain, because the change in DCG from a grade error then depends only on the size of the error and the rank position, not on the original

grade. This is what lets us write the difference between a system’s score under human grades and under LLM grades as a simple inner product (Equation 4.5).

Every system’s score depends on which grades are treated as correct, so changing them changes the scores. A system that placed a passage at the top gains if the new correct grade calls it relevant, and loses if the new grade calls it irrelevant. A system that never retrieved that passage in the top-10 is unaffected by a change.

We want to understand exactly what happens when the grade of one query-passage pair changes from the human grade to the LLM grade, and the two are not the same. That disagreement is what we call a grade error, and the question is how it changes a system’s score.

Both the numerator and the denominator of equation 4.2 change when the grades change, because the best possible score, the denominator, is recomputed from the new grades. The formula can be decomposed into two parts. Write $Z_h(q)$ for the best possible score of Equation 4.1 when the human grades are treated as correct, and $Z_L(q)$ for the same quantity when the LLM grades are. The difference in nDCG of system s when the grades of the test collection change from human to LLM splits exactly into

$$\Delta \text{nDCG}_s(q) = \frac{\text{DCG}_s^L(q)}{Z_L(q)} - \frac{\text{DCG}_s^h(q)}{Z_h(q)} = \text{DCG}_s^h(q) \left(\frac{1}{Z_L(q)} - \frac{1}{Z_h(q)} \right) + \frac{\Delta \text{DCG}_s(q)}{Z_L(q)} \quad (4.3)$$

The first term rescales every system’s human DCG by the same constant, so it preserves the ordering of systems ~~and cannot move one past another~~. The second part is the only one that can produce different shifts for different systems and therefore move systems past each other, and it is the one the rest of this section works with.

Consider a query with n_q graded passages and m_q submitted system runs, where each run is one system’s ranking of passages for that query. For each query we build an error vector $\epsilon_q \in \mathbb{R}^{n_q}$, that ~~holds one number per query passage~~, the LLM grade minus the human grade. A positive value means the LLM graded that pair higher than the human did.

Additionally, for each query and each system, we build a weight vector $\mathbf{w}_{s,q}$, which holds one number per passage, recording how much that passage contributes to the score of system s on query q . As Equation 4.1 shows, a passage ranked first contributes more than the same passage ranked tenth, while a passage the system did not place in its top-10 contributes nothing.

$$w_{s,q,p} = \begin{cases} 1/\log_2(\text{rank}_s(q,p) + 1), & \text{rank}_s(q,p) \leq 10, \\ 0, & \text{otherwise.} \end{cases} \quad (4.4)$$

With these two vectors, the numerator of the second term of Equation 4.3 is a single inner product. When we change the grades from human to LLM, each pair changes the score of system s by its error times its weight, and the total change is the sum over passages:

$$\Delta \text{DCG}_s(q) = \sum_{i=1}^{10} \frac{g_L(p_{s,i}) - g_h(p_{s,i})}{\log_2(i+1)} = \sum_p w_{s,q,p} \epsilon_p = \mathbf{w}_{s,q}^\top \epsilon_q \quad (4.5)$$

Now take two retrieval systems s and t and compare how differently their scores change when the grades on test collection for query q change.

$$\Delta\text{DCG}_s(q) - \Delta\text{DCG}_t(q) = (\mathbf{w}_{s,q} - \mathbf{w}_{t,q})^\top \boldsymbol{\varepsilon}_q = \sum_p (w_{s,q,p} - w_{t,q,p}) \varepsilon_p.$$

Each passage contributes to the sum through the difference in the weight the two systems give it. If both systems placed passage p at the same rank or if neither placed it in the top ten, that difference is zero and the passage does not impact the sum entirely, whatever its error.

This means that when we change the grades in the test collection, a grade error committed by an LLM changes two systems' scores differently, and so creates the possibility of one overtaking the other, only when the two systems rank that passage differently. If both systems rank it the same way, a large error still changes each system's score, but by the same amount, so neither can overtake the other.

The denominator, $Z_L(q)$, of the second term of Equation 4.3 is the same for every system on the query, so it scales **all the shifts** by one common factor and cannot ~~move one system past another.~~

What reorders the systems is not the shift itself but the fact that different systems shift by different amounts. If every system moved by the same amount, the ranking would be untouched. A natural way to measure how much the grade change disturbs the ranking is therefore the variance of $\Delta\text{DCG}_s(q)$ across systems. The error vector does not change from one system to another, since it is just the difference between the LLM and human grades for the passages of that query, so the only source of variation is the weight vector. Since the shift is a fixed linear combination of that vector, the variance is

$$\text{Var}_s[\Delta\text{DCG}_s(q)] = \text{Var}_s[\mathbf{w}_{s,q}^\top \boldsymbol{\varepsilon}_q] = \frac{1}{m_q} \sum_s (\tilde{\mathbf{w}}_{s,q}^\top \boldsymbol{\varepsilon}_q)^2 = \boldsymbol{\varepsilon}_q^\top \left[\frac{1}{m_q} \sum_s \tilde{\mathbf{w}}_{s,q} \tilde{\mathbf{w}}_{s,q}^\top \right] \boldsymbol{\varepsilon}_q = \boldsymbol{\varepsilon}_q^\top C_q \boldsymbol{\varepsilon}_q$$

where $\tilde{\mathbf{w}}_{s,q} = \mathbf{w}_{s,q} - \bar{\mathbf{w}}_q$.

The matrix in the middle is the covariance of the weight vectors across systems, and it is obtained as follows. Let $\bar{\mathbf{w}}_q$ be the average weight vector across the m_q systems. Subtracting $\bar{\mathbf{w}}_q$ from each system's weight vector leaves only the part where that system differs from the average. Averaging the outer product of those **differences creates a matrix with one row and one column per passage.**

$$C_q = \frac{1}{m_q} \sum_s (\mathbf{w}_{s,q} - \bar{\mathbf{w}}_q)(\mathbf{w}_{s,q} - \bar{\mathbf{w}}_q)^\top \quad (4.6)$$

The **diagonal entry** $C_{q,pp}$ is the variance across systems in the weight placed on passage p . $C_{q,pp}$ equals zero when all systems rank passage p the same way. The off-diagonal term $C_{q,pr}$ is the covariance between passages p and r and it shows whether systems tend to treat both passages the same way. **A positive value means that systems ranking passage p near the top usually rank passage r near the top as well, while systems that ignore p also tend to ignore r** Most entries of every weight vector are zero, because nDCG@10 reads only ten passages while a query's pool typically has two hundred or more passages.

Putting the two together gives a measure of the **damage** the LLM grades on query q cause to the ranking of systems:

$$D_q = \text{Var}_s[\Delta\text{DCG}_s(q)] = \boldsymbol{\varepsilon}_q^\top C_q \boldsymbol{\varepsilon}_q \quad (4.7)$$

Damage to the system ranking requires two conditions to hold at the same time: there must be a grading error (so that ϵ_q is non zero for that passage) and the systems must treat the passage differently (so C_q has a non-zero entry there).

Consider the following example. Take two retrieval systems being evaluated, one query with three passages the systems must rank, and a top-two cutoff, meaning that each system contributes only its two highest ranked passages to the score. To evaluate and rank systems we use nDCG@2, which gives more weight to passages the system ranked higher. In this example, and using equation 4.4, a passage ranked first has weight 1 and one ranked second has weight $1/\log_2 3 \approx 0.63$. System A returns p_1, p_2 and system B returns p_3, p_2 . Then

$$\mathbf{w}_A = (1, 0.63, 0), \quad \mathbf{w}_B = (0, 0.63, 1), \quad C = \begin{pmatrix} 0.25 & 0 & -0.25 \\ 0 & 0 & 0 \\ -0.25 & 0 & 0.25 \end{pmatrix}.$$

The two systems agree exactly about p_2 and disagree completely about p_1 and p_3 .

We will analyse three cases and each can be checked two ways: by computing the change in each system’s score from Equation 4.5 and comparing the two, or by evaluating the damage caused to the ranking of systems with Equation 4.7.

If the LLM grades p_2 two points too high, $\epsilon = (0, 2, 0)$. Both systems gain $0.63 \times 2 = 1.26$, so their score moves by the same amount and neither can overtake the other. Our damage formula gives $\epsilon^\top C \epsilon = 4C_{22} = 0$. A serious error that yields no consequence, because both systems rank p_2 the same way.

If the LLM grades p_1 one point too high, $\epsilon = (1, 0, 0)$. System A gains $1 \times 1 = 1$ and system B gains nothing, since $w_{B,1} = 0$. The two systems move apart and the damage is $\epsilon^\top C \epsilon = 1^2 \times C_{11} = 0.25$.

If the LLM grades both p_1 and p_3 one point too high, $\epsilon = (1, 0, 1)$. System A gains 1 through p_1 and system B gains 1 through p_3 , so again they move by the same amount. The damage is $\epsilon^\top C \epsilon = C_{11} + 2C_{13} + C_{33} = 0.25 - 0.50 + 0.25 = 0$. Both passages carry an error and both are ranked differently by the two systems, yet together they cause no damage at all.

Until now, we studied the effect of changing one query’s grades, but systems are ranked by their mean nDCG across all queries in the test collection, so what matters is how all the D_q values combine. Two queries with large D_q can reinforce each other if they inflate the same systems or cancel out if they inflate different ones. So it is not correct to sum D_q values.

Two properties follow. First, C_q comes entirely from the submitted system runs and it records which passages the systems disagree about. That depends on which retrieval systems are being evaluated. The LLM never sees the runs, so nothing in its grades or probabilities can tell it whether a passage matters to the ranking of systems. Second, because C_q has off-diagonal terms, correcting one passage can expose the effect of another, so the value of fixing one error depends on which other errors remain.

4.2 The placement of errors across passages

In section 4.1, we defined D_q , which measures how differently the systems are shifted by the errors on one query q .

Now we focus on the analysis of both components that affect damage. The vector ϵ_q assigns

one number to each passage, representing the difference between the LLM grade and the human grade. It shows which passages have errors and the size of those errors. The matrix C_q comes entirely from the submitted system runs, capturing the disagreement among systems on each passage. If all systems rank a passage first, that passage contributes nothing to damage, because an error there affects all systems’ scores identically.

Damage therefore depends on two factors happening together: a grading error and disagreement among systems on that passage.

Per-query agreement – the Pearson correlation between human and LLM grades for a given query – reflects only the error component. It ignores how systems rank each passage. Consequently, queries whose LLM grades do not agree much with the human grades may not be the ones that affect the ranking of systems most.

We still do not know, however, whether the two components of damage are related. Given the errors the LLM actually made, did those errors happen to land on the passages the systems weight most differently? If so, the two components would not be truly separate: a signal that finds likely errors would also, by accident, find high-leverage passages.

To test this, we shuffle the error vector ϵ_q . By moving the values to different random positions, the same errors now land on different passages, and we then recompute the ranking of systems. We restrict this shuffling to passages that appear in at least one system’s top ten, since passages outside this set cannot change any score. If the actual error placement is special, it will cause more damage than a random assignment. On the other hand, if the two components are unrelated, and the passages the systems rank differently are not the ones the LLM tends to misgrade, then the real placement will not create more damage than a random one and the ratio between them will be close to one. We run 1000 permutations per year and in each draw every query is permuted independently. The total score of each system is recomputed and T_s is system s ’s total score shift relative to the all human ranking. The statistic is $\text{Var}_s[T_s]$, the spread of those shifts across systems and we report the observed value divided by the mean of the 1000 permuted values, meaning that a ratio above one means the real placement is more damaging than random.

The results show that the true placement of errors is 7 to 18 times more damaging than the same errors moved to random passages, meaning that the errors the LLM actually made land on passages that matter more to the ranking of systems than a random passage would. However, this test has a weakness, since the shuffle can move an error to a passage with a different human grade. The LLM’s errors are not spread evenly across the grades: it uses grade 1 for 1.0% of passages against 22.3% for the assessors, and grade 2 for 46.6% against 13.4% (Section 3.2). A free shuffle therefore moves errors onto grade 0 or grade 3 passages, where the LLM rarely fails.

In the stricter version, instead of shuffling errors freely across all passages, we shuffle only within passages that share the same human grade. A grade 1 error can move to another grade 1 passage but never to a grade 0 or grade 3 passage. This way every habit of the LLM that depends on the human grade is preserved, including the confusion counts and pair-level agreement of each query.

Displacement spread, observed / null mean	2019	2020	2021	2022	2023
Free shuffle	11.3	18.0	7.0	13.2	16.9
Within-grade shuffle	1.15	0.88	0.67	0.37	0.35

Table 5: The real error placement against two permutation nulls, 1000 permutations each.

In this second experiment, no positive relation survived and the real placement now does 0.35

to 1.15 times the damage of a random reassignment, and in four collections it does less.

When errors are shuffled only among passages with the same human grade, the LLM’s errors do not land on the passages that matter most to the ranking of systems, and in four of the five years, they actually land on passages that matter less. We offer no explanation as to why this happens.

4.3 Damage concentration within queries

Until now, we changed the grades of every passage in a query at once, but we can also choose specific pairs from different queries to correct and keep the human grades on the rest. We move therefore from working at the query level to the pair level, and ask whether the damage the LLM grades cause on a query is spread evenly over its more than two hundred passages or concentrated on a handful of them.

Separating Equation (4.7) into its diagonal and off-diagonal parts gives

$$D_q = \boldsymbol{\varepsilon}_q^\top C_q \boldsymbol{\varepsilon}_q = \sum_p \sum_r \varepsilon_p \varepsilon_r C_{q,pr} = \sum_p \varepsilon_p^2 C_{q,pp} + 2 \sum_{p < r} \varepsilon_p \varepsilon_r C_{q,pr} \quad (4.8)$$

The first sum gives each passage an individual contribution, large only when the LLM’s error there is large and the systems weight that passage differently. The second sum runs over pairs of passages and represents the interaction between them: two errors can reinforce each other or cancel out. The factor 2 appears because C_q is symmetric, so each pair is counted twice in the double sum.

Suppose two passages of a query are weighted the same by every system and the LLM grades one a point too high and the other a point too low. Each error moves every system’s score, but in opposite directions, so together they leave the ranking untouched and the query does no damage at all. Now correct either one, by putting back the human grade in place of the LLM one, and the other creates more damage than before the correction.

This interaction stops us from simply correcting pairs first only by their first term contribution. So in this experiment we do not rank all the passages by their first-term contribution and correct in that order. We correct one pair at a time, always picking the one that helps most at that step.

Consider a query q with n_q passages. Write $\mathbf{r}$ for the errors still uncorrected at the current step, so $\mathbf{r}$ starts equal to $\boldsymbol{\varepsilon}_q$ and gains a zero entry with each correction. The damage still present is computed from Equation 4.7 using $\mathbf{r}$, giving $\mathbf{r}^\top C_q \mathbf{r}$, and correcting passage p changes the damage by:

$$\text{reduction}(p) = 2r_p (C_q \mathbf{r})_p - C_{q,pp} r_p^2, \quad (4.9)$$

The identity follows from expanding the quadratic form. Correcting passage p replaces $\mathbf{r}$ with $\mathbf{r} - r_p \mathbf{e}_p$, where $\mathbf{e}_p$ has a one in position p and zeros elsewhere, so the remaining damage becomes

$$(\mathbf{r} - r_p \mathbf{e}_p)^\top C_q (\mathbf{r} - r_p \mathbf{e}_p) = \mathbf{r}^\top C_q \mathbf{r} - 2r_p (C_q \mathbf{r})_p + r_p^2 C_{q,pp}, \quad (4.10)$$

using $\mathbf{e}_p^\top C_q \mathbf{r} = (C_q \mathbf{r})_p$ and $\mathbf{e}_p^\top C_q \mathbf{e}_p = C_{q,pp}$. Subtracting this from the damage before the correction gives Equation (4.9).

To test whether the damage is evenly spread across passages or concentrated in some, we take the full error vector, evaluate Equation 4.9 for every uncorrected passage and correct the one that gives the largest reduction. This repeats until no single correction reduces the damage any further. The stopping rule leaves some damage standing, because some groups of errors only reduce the damage collectively, so no passage in such a group can be corrected on its own without increasing the damage.

We register for each query the number of pairs that need to be corrected to remove 50 and 80 percent of that query’s removable damage, each as a fraction of the number of passages. We take the median over the queries of a year and the ranges presented in the table below are the ranges of those five medians.

Median per query	2019	2020	2021	2022	2023
Passages for half the damage	3	3	5	4	3
As a share of passages	2.6%	3.1%	4.8%	2.3%	2.4%
Share for 80% of the damage	7.9%	7.8%	15.4%	7.9%	6.8%

Table 6: Damage inside a query is carried by a few passages.

This greedy experiment removes half of the damage with a median of three to five corrections per query, 2.3 to 4.8 percent of the judged passages and 80 percent with 7 to 15 percent of them. This result motivates working at the passage level, since a small set of passages creates most of the damage in the ranking of systems.

This is an oracle result. The greedy algorithm uses ε_p , the difference between the LLM grade and the human grade, which is only known once a human assessor has graded the passage. It cannot be used to decide which passages to send in the first place. It shows that there is a cheap way to correct at the passage level, but we still do not have a policy, that does not use the human grades, to choose which passages to send to a human assessor first.

We cannot use the greedy oracle to choose passages, because it needs the true errors. However, we can approximate it. The interaction between errors is small, so we drop it and score each passage on its own by the product of two factors, the squared error ε_p^2 and the diagonal entry $C_{q,pp}$.

The squared error is the error the LLM actually made, which needs the human grade we do not have, so we replace it with an estimate. We call $C_{q,pp}$ the leverage of passage p . It is the variance across systems of the weight they give that passage, and it measures how much an error there could matter. Leverage depends on which systems are compared, so it is not an intrinsic property of a passage. Combining the two gives the selection policy we propose, which orders query-passage pairs by the order in which they should be sent to a human assessor.

$$\text{score}(p) = \mathbb{E}[\varepsilon_p^2] C_{q,pp} \quad (4.11)$$

The passage most worth sending to a human assessor is the one that is both likely to be misgraded and likely to matter to the ranking of systems.

4.4 Correcting the ranking at the passage level

As we saw in the last subsections, correcting whole queries is expensive, because the human assessor grades every passage of that query when only a small set of passages carries most of

the damage to the ranking. Correcting single passages, in turn, is cheap and an oracle that knows every error needs only three to five corrections per query to remove half of that query’s damage. This subsection tests whether it is possible to find a policy that efficiently finds those pairs.

The setup repeats the experiment of Section 3.5, with the query-passage pair replacing the query. We start with LLM grades on every judged pair. Sending one query-passage pair to a human assessor replaces its LLM grade with the human grade, which sets that pair’s error to zero.

The budget is now the fraction of ~~pairs~~ sent to a human, rather than the fraction of queries. Pairs not sent keep their LLM grade, so one query can hold human and LLM grades at the same time. A policy is an ordering over pairs, and the question is what budget a policy needs before the ranking it produces agrees with the original ranking, the one produced when we evaluate the systems using the fully human-graded test collection.

Each policy chooses and sends one percent of all pairs at a time to the human assessor. After each batch of pairs sent to the human assessor, we rebuild the test collection, rescore every retrieval system with nDCG@10 and compare the resulting ranking of systems with the reference ranking. Any quantity the policy uses is recomputed at each batch. The reported metric is the smallest budget at which Kendall τ @20 reaches 0.95. This metric is used since, as Clarke and Dietz (2025) showed, **it is not problematic to obtain good results over all systems**. The problem appears when we try to distinguish between the top systems. A test collection graded entirely by an LLM produces a ranking of retrieval systems that agrees with the human ranking at a τ @20 of 0.51 (Clarke and Dietz, 2025).

Section 4.2 showed that damage to the ranking of systems can be decomposed in two components: the error ~~committed by~~ the LLM on that passage and the leverage, $C_{q,pp}$, defined as the variance across systems of the weight **they give** that passage. A selection policy can try to estimate the error component, the leverage component, both or neither. A policy that uses the error component estimates in some way whether the LLM grade for a passage is correct. A policy that uses the leverage component tries to identify which passages are more important in distinguishing between systems. As a baseline against our proposed methods, we take the strongest published method in each of these groups. For the error component that is LARA (Takehi et al., 2025). For the leverage component it is move-to-front pooling (Cormack et al., 1998), one of the methods LARA itself compares against.

The leverage component has a more recent work worth discussing. Minimal test collections (Carterette et al., 2006) starts with no grades at all. It chooses which pairs to send first to a human assessor by how much the grade would change the gap between two systems. A passage that both systems rank at the same position has weight zero and will not change the order of the two systems, which is a similar idea to the leverage component of Equation 4.7.

The following policies are tested:

- **Random.** This selection policy does not use any of the components. It is the simplest policy and sends pairs to a human assessor in random order. All reported results are averaged over 100 independent draws.
- **LARA.** The selection rule is to send first the pairs whose grades are most likely to be wrong. For each pair, Takehi et al. (2025) take the difference between the two highest probabilities the LLM produced over the four grades, and call it the margin. A small margin means the LLM was nearly split between two grades, while a large margin means

it settled clearly on one. LARA sends pairs to the human assessor in increasing order of margin. We reproduce it as the strongest baseline policy that uses the error component. It is also important to note that LARA was built to minimise the number of wrong LLM grades in the collection and not to preserve the ranking of the top-20 systems.

- **Move-to-front pooling (MTF)** Move-to-front pooling (Cormack et al., 1998) does not give each passage a fixed score, but rather selects them one at a time. For each query it keeps a priority for every submitted system and takes the next passage from the system at the top of that order, choosing the highest ranked passage that has not been graded yet. If that passage is graded as relevant, the same system stays at the top and gives the next one. If it is not, the system drops down the order. A passage already graded for another system still updates the priority but costs no budget. The effort therefore follows whichever systems are currently producing relevant passages. MTF reads the submitted system runs and the human grades it has collected so far, but does not try to estimate the error committed by the LLM on a pair.
- **Leverage-based policy.** This is the first proposed policy. Leverage $C_{q,pp}$ is the variance across systems of the weight they give passage p , computed from the system runs alone. Of the two components in Equation 4.11, this policy tests using only leverage alone. Pairs are sent to a human assessor in descending order of leverage, meaning that pairs that could damage the ranking of systems more are sent first. This method starts by having an LLM grade all pairs. Using this fully LLM-graded test collection, the retrieval systems are evaluated and a ranking of systems is created from their scores. We then build the covariance matrix over the twenty systems currently ranked highest, according to our LLM grades, which matches the measure we report. The rule is adaptive: sending pairs to the human assessor and correcting the errors the LLM made on these changes the ranking of systems, which can change which twenty systems are ranked highest, which can also change which passages have the highest leverage for the next batch. It is not one fixed sorting computed once at the start. At each batch, every system is evaluated on the currently mixed test collection, the top twenty is read from that ranking and C_q is rebuilt for every query over those twenty systems.
- **Leverage-Confidence Based (LCB)** This is the main proposed policy, derived from Equation 4.11. It multiplies the two components, leverage and error, sending pairs to a human assessor in descending order of $\mathbb{E}[\varepsilon_p^2] C_{q,pp}$.

The expected squared error is not known, since we do not have the human grade, so it has to be estimated. The LLM already reports a probability for each of the four grades, so we use those probabilities directly. If $g_L(p)$ is the grade the LLM gave and $g_h(p)$ the unknown human grade, the estimate is

$$\mathbb{E}[\varepsilon_p^2] = \sum_{h=0}^3 P(g_h(p) = h) (g_L(p) - h)^2 \quad (4.12)$$

A pair where the LLM spread its probability across grades far from the one it chose gets a high error estimate and a pair where it put almost everything on one grade gets a low one.

We know from Takehi et al. (2025) that the probabilities an LLM outputs are not calibrated, which leaves two possibilities. The first uses them as they are, treating the LLM’s own probabilities as if they were probabilities over the human grade. The second replaces the LLM’s probabilities with a distribution over human grades learned from the other queries of the same year. We group the pairs of a year by the grade the LLM gave and

by how confident it was, measured by the largest of its four probabilities and split at its quantiles into three groups. Within each group we count how often the human assessor gave each of the four grades, and normalising those counts gives $P(g_h(p) = h)$ for Equation 4.12. The counts for a given query are taken from all other queries of that year, so a query never informs its own error estimate, and a group holding fewer than twenty pairs is merged with the others sharing the same LLM grade. This calibrated version needs some human grades before it can start, which does not prevent its use.

The leverage is computed as in the leverage-based policy. An LLM grades all pairs and every system is scored on those grades. The covariance matrix is built over the twenty systems ranked highest, using LLM grades. For each query-passage pair the product of the two components is computed and pairs are sent to the human assessor in descending order of that product. Both quantities are recomputed after each batch is sent to human

Policy	2019	2020	2021	2022	2023
Random	99	98	99	93	97
LARA	40	48	63	71	40
Move-to-front pooling	23	33	67	28	42
Leverage Policy	15	14	30	11	15
Leverage-Confidence Based (LCB) raw	15	15	29	12	16
Leverage-Confidence Based (LCB) calibrated	13	10	28	9	14

Table 7: Percentage of query-passage pairs sent and corrected by a human assessor, before $\tau@20$ reaches 0.95. The uncertainty is reported on the paired advantage over random in Figure 6

From Table 7, the random baseline always needs to send more than 90 percent of the pairs to the human assessor to reach an almost perfect ordering. LARA improves on random and needs 40 to 71 percent of all pairs. Move-to-front pooling needs 23 to 67 percent. While MTF is better on average than LARA, 39 percent against 52, the comparison is not clear, since LARA performs better in 2021 and 2023 but loses by a large margin in 2022.

In turn, our first proposed method, the leverage-based policy, needs only 11 to 15 percent in four of the five collections, less than LARA and MTF in every year.

The two LCB policies match or slightly improve on leverage in every collection. When we add the LLM’s own estimate of error to the leverage component it does not worsen and in some years saves budget, though the paired differences do not exclude zero at this sample size. The practical conclusion is that leverage alone is responsible for most of the gains and the error component adds a small improvement. It is important to keep in mind that the numbers in Table 7 depend on the choice of 0.95 as a threshold. We also computed the difference in $\tau@20$ between policy and random selection at each budget on the same queries and resampled the queries to obtain an interval on that difference. The advantage becomes distinguishable from zero at 8 to 24 percent of the budget for MTF, the leverage-based policy and the two LCB variants, and at 47 to 87 percent for LARA, which never reaches that point in 2022.

Regarding this final experiment, one limit should be mentioned. The advantage claimed is specific to the top of the ranking, which is where this thesis directs its focus, since that is the problem raised by Clarke and Dietz (2025). In Appendix B we also provide the values of the agreement between the whole rankings, where the policies are much closer and leverage does not improve on random there.

Figure 5 shows the evolution of the Kendall τ agreement between the two rankings of the top twenty systems as the budget increases. The policies that use the leverage component rise

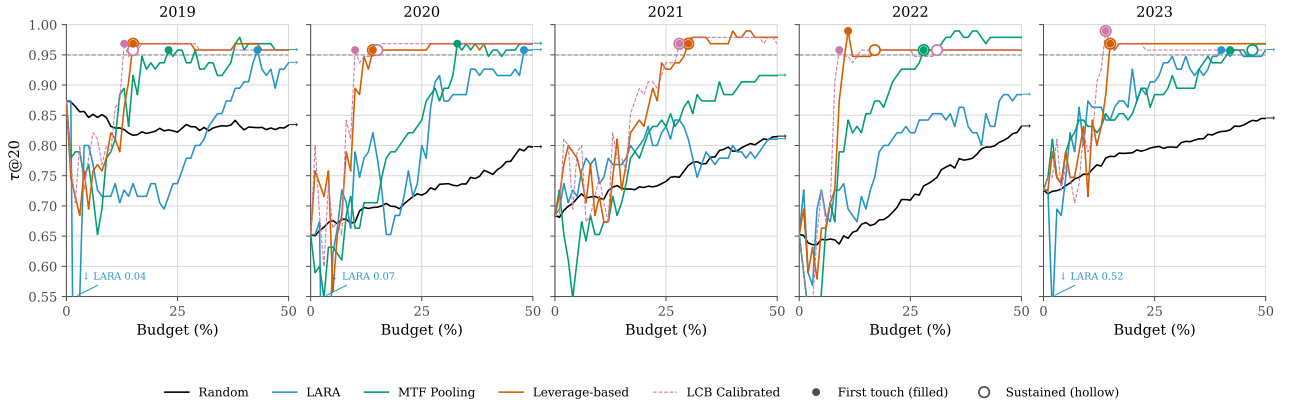


Figure 5: Agreement between top systems, measured as the Kendall Tau agreement over the twenty best systems against the proportion of pairs sent to a human assessor. One panel for each year, with the budget axis cut at 50 percent. Filled markers show the first budget at which $\tau@20$ reaches 0.95, while hollow markers the budget from which it stays there. The dashed grey line marks 0.95.

sharply in the first ten percent of the budget and then flatten near 0.95, while LARA climbs slowly and usually stays below them. Random selection is, as one could expect, the lowest curve in every collection and does not reach a τ of 0.95 before 50 percent of the pairs have been sent to the human assessor. There is also a noticeable advantage of LCB over MTF, especially in 2020 and 2021.

For each policy we report two markers, filled and hollow. The filled marker is the first budget at which the policy reaches 0.95 and the hollow one is the budget from which it stays at or above 0.95 for the rest of the experiment. This matters because agreement over the top twenty is not monotone in budget, so a policy can reach 0.95 and then drop below it as the budget grows.

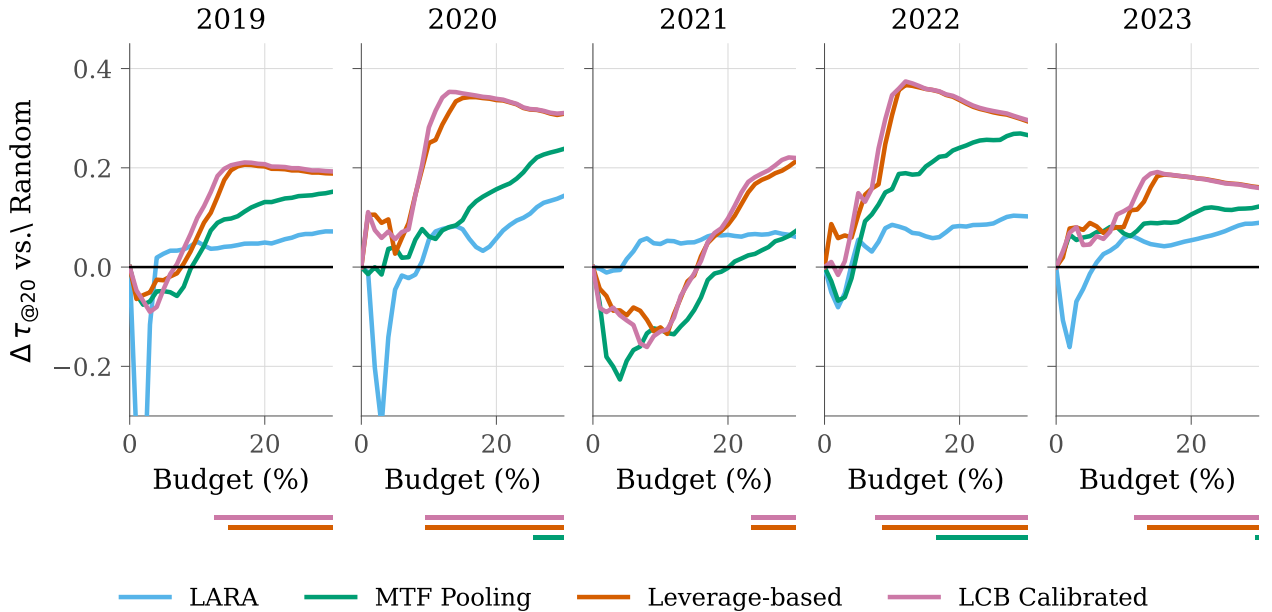


Figure 6: Difference in $\tau@20$ between each policy and random selection at the same budget, with the budget axis cut at 30 percent. Under each panel, we included bars that indicate the budgets at which the 95 percent interval, obtained by query resampling, excludes zero.

In Figure 6, the vertical axis represents the gain each policy has over the random baseline, defined as the difference in Kendall τ at each budget level. Across all four policies, gains generally begin below zero in the first few percent of the budget, where the policy has not yet produced any measurable effect and the differences reflect noise. Around 15 percent of the budget, policies that use the leverage component separate from the remaining two, reaching peaks between 0.2 and 0.35, depending on the collection. The visible exception is the year 2021, where the leverage-based policy and LCB only rise above zero at around 15 percent of the budget and become distinguishable from zero after 20 percent.

The bars under each plot show where the advantage can be distinguished from zero. At each budget we take the difference in $\tau@20$ between the policy and random selection on the same queries, then resample the queries with replacement 1000 times and recompute that difference on each sample. The bar is drawn at the budgets where the interval between the 2.5 and 97.5 percentiles of those 1000 values excludes zero. Leverage and the product reach that point at 8 to 24 percent of the budget, and LARA at 47 to 87 percent, never reaching it at all in 2022. The two policies that read the runs also fall back towards the others at the right of each panel, since at high budgets every policy has corrected most of what matters and the orderings converge.

5 Discussion and Conclusion

The thesis started by asking whether the LLM knows where it is uncertain or likely incorrect in a grade, and if that can be predicted from its own probabilities. We also noted that sending whole queries for human assessors to grade is inefficient, and instead it is possible to send individual passages that we are uncertain about. To decide which passages to send, we analysed how a single error on a grade damages the order of systems in the ranking. Equation (4.7) decomposes the damage created by an error into two components: the error component, that measures how wrong the grade is, and the leverage component, that measures how differently the systems rank a specific passage. A wrong grade on a passage that every system ranks the same way does not affect the ordering, while a small error on an important passage where systems disagree can affect a lot. Using this information, we build a policy that prioritises passages we are uncertain about and that are ranked differently by the submitted systems. In the end, our main proposed policy obtains a ranking of systems very close to the one built from human grades alone, with trained human assessors grading only 9 to 28 percent of the pairs.

Three limitations of this work should be stated. First, every experiment was tested using only one LLM (Llama-3.1-8B-Instruct), one instruction prompt from UMBRELA and one family of test collections (TREC Deep Learning from 2019 to 2023). This questions if another LLM would be better or worse at predicting its own confidence on a grade, which could most of the work presented. Second, the damage decomposition in Equation 4.7 is exact only under two conditions. The first is that the numerator of each term in the DCG sum is the grade itself, which we called linear gain in Section 4.1, rather than the $2^g - 1$ that TREC uses. The second is that the denominator is held fixed, so that we work with the unnormalised score shift. Every ranking in the thesis is scored with the full nDCG formula, so the reported budgets do not depend on the approximation. The ordering the policy produces, on the other hand, does, since it is derived from the linear case. If our policy used exponential gain, the same grade error would change the score by a different amount depending on the original grade, so the ordering would be different and possibly even more efficient. Lastly, our proposed policy uses the submitted runs to choose which passages are most important to be graded correctly by humans. This raises the question of whether the final test collection, that includes LLM and human grades, can also be used to score and rank a new retrieval system whose run was not

submitted and considered.

There are also some future directions that emerge from this work. The most important question is whether the methods introduced extend to unseen systems. If a new system retrieves mostly the same passages as the existing runs, the passages we corrected first will also apply. Measuring this would help us understand if this method could actually be used to build test collections in a more efficient way. A second direction is to apply the same methods of this thesis to stronger and larger LLMs. The exponential evolution of LLMs in the last years, has created an enormous gap between state-of-the-art industry models and open-source models. The interesting question is ~~we introduce is~~ not whether a larger model grades more accurately - we can expect that it does - but whether if it is able to self-report its confidence more accurately. If a stronger model uses all four grades, the thesis predicts specific consequences: the static ceiling rises, the label wall thins, passage-level confidence becomes more informative, and the human budget drops. Finally, deriving the damage decomposition for exponential $nDCG$ gain would close the second limitation above and may reveal that top-ranked passages deserve even more priority than the current policy gives them.

References

- Alaofi, M., P. Thomas, F. Scholer, and M. Sanderson (2024). LLMs can be fooled into labelling a document as relevant: best café near me; this paper is perfectly relevant. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, pp. 32–41. ACM.
- Aslam, J. A. and R. Savell (2003). On the effectiveness of evaluating retrieval systems in the absence of relevance judgments. In *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, Toronto, Canada, pp. 361–362. ACM.
- Bailey, P., N. Craswell, I. Soboroff, P. Thomas, A. P. de Vries, and E. Yilmaz (2008). Relevance assessment: are judges exchangeable and does it matter? In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 667–674. ACM.
- Balog, K., D. Metzler, and Z. Qin (2025). Rankers, judges, and assistants: towards understanding the interplay of LLMs in information retrieval evaluation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 3865–3875. ACM.
- Carterette, B., J. Allan, and R. Sitaraman (2006). Minimal test collections for retrieval evaluation. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 268–275. ACM.
- Clarke, C. L. A. and L. Dietz (2025). LLM-based relevance assessment still can’t replace human relevance assessment. In *Proceedings of the Tenth International Workshop on Evaluating Information Access (EVIA 2025), a Satellite Workshop of the NTCIR-18 Conference*. arXiv preprint arXiv:2412.17156.
- Cormack, G. V., C. R. Palmer, and C. L. A. Clarke (1998). Efficient construction of large test collections. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 282–289. ACM.
- Craswell, N., B. Mitra, E. Yilmaz, and D. Campos (2021). Overview of the TREC 2020 deep learning track. In *Proceedings of the Twenty-Ninth Text REtrieval Conference (TREC 2020)*.
- Craswell, N., B. Mitra, E. Yilmaz, D. Campos, and J. Lin (2022). Overview of the TREC 2021 deep learning track. In *Proceedings of the Thirtieth Text REtrieval Conference (TREC 2021)*.
- Craswell, N., B. Mitra, E. Yilmaz, D. Campos, J. Lin, E. M. Voorhees, and I. Soboroff (2023). Overview of the TREC 2022 deep learning track. In *Proceedings of the Thirty-First Text REtrieval Conference (TREC 2022)*.
- Craswell, N., B. Mitra, E. Yilmaz, D. Campos, and E. M. Voorhees (2020). Overview of the TREC 2019 deep learning track. In *Proceedings of the Twenty-Eighth Text REtrieval Conference (TREC 2019)*.
- Craswell, N., B. Mitra, E. Yilmaz, H. A. Rahmani, D. Campos, J. Lin, E. M. Voorhees, and I. Soboroff (2025). Overview of the TREC 2023 deep learning track. *arXiv preprint arXiv:2507.08890*.

- Cronen-Townsend, S., Y. Zhou, and W. B. Croft (2002). Predicting query performance. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 299–306. ACM.
- Faggioli, G., L. Dietz, C. L. A. Clarke, G. Demartini, M. Hagen, C. Hauff, N. Kando, E. Kanoulas, M. Potthast, B. Stein, and H. Wachsmuth (2023). Perspectives on large language models for relevance judgment. In *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*, pp. 39–50. ACM.
- MacAvaney, S. and L. Soldaini (2023). One-shot labeling for automatic relevance estimation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2230–2235. ACM.
- Merlo, S., S. Marchesin, G. Faggioli, and N. Ferro (2025). A cost-effective framework to evaluate LLM-generated relevance judgements. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pp. 2115–2126. ACM.
- Soboroff, I. (2025). Don’t use LLMs to make relevance judgments. *Information Retrieval Research* 1(1), 29–46.
- Soboroff, I., C. Nicholas, and P. Cahan (2001). Ranking retrieval systems without relevance judgments. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 66–73. ACM.
- Takehi, R., E. M. Voorhees, T. Sakai, and I. Soboroff (2025). LLM-assisted relevance assessments: when should we ask LLMs for help? In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 95–105. ACM.
- Thomas, P., S. Spielman, N. Craswell, and B. Mitra (2024). Large language models can accurately predict searcher preferences. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1930–1940. ACM.
- Unell, A., N. Dullerud, N. Boneh, M. Jagadeesan, T. Hashimoto, N. Shah, and S. Koyejo (2026). Metric Match: a subset selection approach to evaluating LLM judge reliability. *arXiv preprint arXiv:2606.15029*.
- Upadhyay, S., R. Pradeep, N. Thakur, D. Campos, N. Craswell, I. Soboroff, H. T. Dang, and J. Lin (2024b). A large-scale study of relevance assessments with large language models: an initial look. *arXiv preprint arXiv:2411.08275*.
- Upadhyay, S., R. Pradeep, N. Thakur, N. Craswell, and J. Lin (2024a). UMBRELA: Umbrella is the (open-source reproduction of the) Bing RElevance Assessor. *arXiv preprint arXiv:2406.06519*.
- Vinay, V., I. J. Cox, N. Milic-Frayling, and K. Wood (2006). On ranking the effectiveness of searches. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 398–404. ACM.
- Wang, V., M. J. Q. Zhang, and E. Choi (2025). Improving LLM-as-a-judge inference with the judgment distribution. *arXiv preprint arXiv:2503.03064*.
- Zhou, Y. and W. B. Croft (2007). Query performance prediction in web search environments. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 543–550. ACM.

A Full Feature Inventory

This appendix lists every feature tested in Section 3.4. Each feature produces one value per query and is evaluated by its Kendall τ against the per-query Pearson correlation between LLM and human grades. Features are grouped into three families. Family A reads only the grade distribution the LLM produced for a query. Family B perturbs the input and measures how much the LLM’s grades change. Family C reads the probability vectors the LLM reports over the four grades. In total, 54 features were evaluated: 10 in Family A, 27 in Family B (9 per operator) and 17 in Family C.

Table 8: Families A and C. Left: static distributional features, computed from the integer grades. Right: generative-internal features, computed from the probability vectors over the four grades.

Family A — Static distributional		Family C — Generative-internal	
Feature	τ	Feature	τ
score_entropy	0.221	fisher_ratio	0.297
score_std	0.200	expected_score_std	0.281
compression_ratio	−0.195	prob_total_variance	0.251
top_heaviness	0.166	mean_pairwise_JSD	0.249
score_iqr	0.120	CWD_ratio	0.230
mean_score	0.086	prob_matrix_eff_rank	0.214
binary_entropy	0.073	dprime_0v23	0.144
frac_zero	−0.041	SSA_full	0.123
discriminability_gap	−0.037	frac_high_conf	0.052
gini_coefficient	−0.020	mean_max_prob	0.051
		conf_std	0.043
		conf_cv	0.034
		mean_prob_margin	0.023
		frac_low_conf	−0.038
		mean_token_entropy	−0.078
		TPL_quadratic	−0.103
		CWD_strict	0.008

Table 9: Perturbation features. Each operator perturbs the passage or prompt three times and summarises how the LLM’s grades change. The nine summary statistics are the same across operators and only the perturbation differs.

Synonym substitution		Sentence permutation		Instruction rewording	
Feature	τ	Feature	τ	Feature	τ
stability_tau	0.169	stability_tau	0.338	stability_tau_pw	0.295
stability_spear	0.167	stability_spear	0.329	stability_tau	0.259
stability_tau_pw	0.166	stability_tau_pw	0.293	stability_spear	0.249
instability_mad	-0.080	mean_score_shift	-0.135	mean_signed_shift	-0.085
instability_var	-0.072	prob_mass_shift	-0.133	mean_score_shift	0.076
mean_score_shift	-0.061	instability_mad	-0.131	entropy_shift	-0.041
prob_mass_shift	-0.058	instability_var	-0.107	prob_mass_shift	0.036
entropy_shift	-0.038	mean_signed_shift	-0.094	instability_mad	-0.036
mean_signed_shift	-0.011	entropy_shift	-0.045	instability_var	-0.014

B Per-Year Correction Tables

The body reports the budget at which $\tau@20$ reaches 0.95 (Table 7). This appendix reports the same experiment under two additional measures: agreement over all systems (τ_{all}) and the largest rank drop of any single system.

Agreement over all systems

Policy	2019	2020	2021	2022	2023
Random	87	76	77	68	77
LARA	49	34	46	37	24
Move-to-front pooling	17	33	37	25	31
Leverage	43	23	59	64	29
LCB raw	30	35	56	57	42
LCB calibrated	37	37	68	58	14

Table 10: Percentage of judged pairs verified before τ_{all} reaches 0.95. The leverage-based policies that dominate on $\tau@20$ (Table 7) require a larger budget here than LARA or move-to-front pooling. The advantage is specific to the top of the ranking.

Largest rank drop

The largest rank drop is the most positions any single system falls relative to the human ranking. At budget zero it ranges from 4 to 17 across years (Table 3). Table 11 reports this quantity at the budget where each policy first reaches $\tau@20 \geq 0.95$.

Policy	2019	2020	2021	2022	2023
Random	2	2	2	5	1
LARA	4	3	3	6	3
Move-to-front pooling	2	4	3	5	1
Leverage	3	9	10	21	4
LCB raw	3	10	17	23	8
LCB calibrated	6	10	14	25	3

Table 11: Largest rank drop of any single system at the budget where each policy first reaches $\tau@20 \geq 0.95$ (Table 7). The leverage-based policies reach high rank correlation over the top twenty while individual systems may still be displaced by up to 25 positions. Random and move-to-front pooling reach the same threshold later but with smaller worst-case displacements, because they have corrected more pairs by that point.

$\tau@20 \geq 0.90$ threshold

The body uses $\tau@20 \geq 0.95$ as the target. Table 12 repeats the comparison at the looser threshold of 0.90.

Policy	2019	2020	2021	2022	2023
Random	90	89	90	78	79
LARA	40	28	57	56	20
Move-to-front pooling	15	32	37	21	29
Leverage	14	12	24	10	15
LCB raw	15	11	21	12	14
LCB calibrated	13	10	21	8	12

Table 12: Percentage of judged pairs verified before $\tau@20$ reaches 0.90. The ordering across policies is the same as at the 0.95 threshold. LCB calibrated reaches 0.90 with 8 to 21% of the pairs.

$\tau@20$ at selected budget levels

Table 13 reports $\tau@20$ at seven budget levels. These values are read from single runs, not averaged over resampling.

Budget	Policy	2019	2020	2021	2022	2023
5%	Random	0.85	0.68	0.71	0.64	0.73
	LARA	0.80	0.62	0.72	0.73	0.75
	MTF	0.77	0.63	0.61	0.56	0.79
	Leverage	0.69	0.56	0.78	0.66	0.79
	LCB raw	0.73	0.68	0.74	0.74	0.80
	LCB cal	0.78	0.66	0.79	0.73	0.77
10%	Random	0.83	0.67	0.71	0.65	0.75
	LARA	0.72	0.79	0.74	0.74	0.87
	MTF	0.79	0.66	0.65	0.79	0.83
	Leverage	0.82	0.89	0.72	0.94	0.72
	LCB raw	0.83	0.80	0.68	0.87	0.78
	LCB cal	0.79	0.96	0.75	0.95	0.79
15%	Random	0.82	0.70	0.73	0.67	0.78
	LARA	0.74	0.78	0.77	0.69	0.84
	MTF	0.92	0.71	0.71	0.85	0.84
	Leverage	0.97	0.96	0.73	0.95	0.97
	LCB raw	0.97	0.96	0.74	0.95	0.91
	LCB cal	0.96	0.96	0.78	0.96	0.97
20%	Random	0.82	0.70	0.73	0.68	0.79
	LARA	0.74	0.68	0.81	0.80	0.93
	MTF	0.93	0.80	0.80	0.88	0.85
	Leverage	0.97	0.96	0.83	0.96	0.97
	LCB raw	0.97	0.96	0.85	0.95	0.98
	LCB cal	0.97	0.97	0.88	0.96	0.97
30%	Random	0.82	0.74	0.77	0.74	0.80
	LARA	0.80	0.87	0.81	0.85	0.93
	MTF	0.93	0.89	0.87	0.96	0.89
	Leverage	0.96	0.97	0.97	0.96	0.97
	LCB raw	0.97	0.97	0.98	0.96	0.97
	LCB cal	0.96	0.97	0.98	0.95	0.96
50%	Random	0.83	0.80	0.81	0.83	0.84
	LARA	0.94	0.96	0.81	0.88	0.96
	MTF	0.96	0.97	0.92	0.98	0.97
	Leverage	0.96	0.97	0.98	0.96	0.97
	LCB raw	0.96	0.97	0.98	0.96	0.98
	LCB cal	0.96	0.97	0.97	0.96	0.96

Table 13: $\tau@20$ at selected budget levels. At 10–15% the leverage-based policies already separate from the others. By 30% the three leverage variants plateau near 0.96–0.98, while LARA and random are still below 0.90 in most years.